

MongoDB en GloboTour: El Catálogo Flexible

Departamento de Informática

3 de junio de 2026

1 MongoDB en GloboTour: El Catálogo Flexible

En **GloboTour**, el catálogo de experiencias crece cada día. Tenemos desde cursos de cocina hasta alquiler de yates. Usamos MongoDB porque nos permite guardar cada experiencia con sus propios campos sin complicaciones.

1.1 Conceptos Clave (Teoría)

Concepto SQL	Concepto MongoDB	Descripción
Tabla	Colección	Agrupación de documentos (no requiere esquema fijo).
Fila	Documento	Registro individual en formato BSON (JSON binario).
Columna	Campo	Par Clave-Valor dentro de un documento.

1.1.1 El dilema del Modelado: ¿Embeber o Referenciar?

A diferencia de SQL (donde normalizamos), en MongoDB tenemos dos opciones:

1. **Embeber (Denormalizar)**: Metemos los datos dentro del documento (ej: los detalles técnicos dentro de la experiencia). Es más rápido para leer.
2. **Referenciar (Normalizar)**: Guardamos un ID de otro documento. Se usa si los datos crecen mucho (ej: miles de comentarios en un post).

¡Ojo con los Tipos de Datos!: En MongoDB, `String` y `Number` **no son lo mismo**. Si guardas el precio como texto, las consultas de «mayor que» (`$gt`) no funcionarán como esperas.

Tip de Depuración: Si tu consulta no devuelve nada:

1. Asegúrate de estar en la base de datos correcta: `globotour`.
2. Comprueba el tipo de dato real con: `typeof`.

1.2 Visualización con Mongo Express (GUI)

1. Abre tu navegador en `http://localhost:8081`.
 2. **Login:** Usuario: `admin` / Password: `pass`.
 3. **Navegación:** Haz clic en la base de datos `globotour` y luego en la colección `experiencias` para explorar los documentos JSON.
 4. **Editor Visual:** Puedes editar el JSON directamente desde la web para probar cambios rápidos sin escribir comandos.
-

1.3 Operaciones Básicas (CRUD)

1.3.1 Inserción de Documentos (Create)

```
// Insertar una nueva experiencia individual
db.experiencias.insertOne({
  nombre: "Cena a ciegas",
  tipo: "Gastronomia",
  precio: 65,
  tags: ["romantico", "sensorial"]
})
```

1.3.2 Consultas de Lectura (Read)

```
// Sacar todas las experiencias
db.experiencias.find()

// Buscar por coincidencia exacta y operadores
db.experiencias.find({ precio: { $in: [25, 45, 80] } })

// Buscar documentos que TIENEN un campo concreto
db.experiencias.find({ detalles: { $exists: true } })
```

1.3.3 Actualización de Datos (Update)

```
// Modificar un campo específico ($set)
db.experiencias.updateOne(
  { nombre: "Cena a ciegas" },
  { $set: { precio: 70, dificultad: "Baja" } }
)

// Añadir un elemento a un array ($push)
db.experiencias.updateOne(
  { nombre: "Cena a ciegas" },
  { $push: { tags: "popular" } }
)
```

1.3.4 Borrado de Datos (Delete)

```
// Borrar un documento concreto
db.experiencias.deleteOne({ nombre: "Cena a ciegas" })
```

1.4 Desafíos de Aprendizaje

1. **Esquema Flexible:** Inserta una nueva experiencia con campos únicos (por ejemplo, un "curso de surf" con un instructor ("Nacho") y materiales incluidos (`material_incluido: ["tabla", "neopreno"]`)). Comprueba que MongoDB la acepta sin necesidad de alterar ninguna tabla.
2. **Proyecciones:** Modifica una consulta `find` para que solo devuelva el nombre del producto, ocultando el `_id`.
3. **Ampliación Avanzada:** Una vez comprendida la base, dirígete al **02-Laboratorio__Agregaciones.** para dominar el Pipeline de Agregación y el análisis complejo de datos.

1.5 Solucionario de Desafíos

Solución Desafío 1: Esquema Flexible

Podemos añadir campos que no existen en otros documentos, como `instructor` o `material_incluido`:

```
db.experiencias.insertOne({
  nombre: "Curso de Surf",
  instructor: "Nacho",
  material_incluido: ["tabla", "neopreno"],
  precio: 40
})
```

Si haces un `db.experiencias.find()`, verás que este documento convive perfectamente con los demás.

Solución Desafío 2: Proyecciones

El segundo parámetro de `find()` permite elegir qué campos mostrar (1) u ocultar (0). El `_id` es el único que se muestra siempre a menos que se oculte explícitamente:

```
// Mostrar solo nombre (1) y ocultar id (0)
db.experiencias.find({}, { nombre: 1, _id: 0 })
```